Adversarial Prompts in Long Contexts

Leon Luo & Ben Kim
University of Chicago
{leonluo, benkim}@uchicago.edu

Project Mentor: Dang Nguyen
University of Chicago

Abstract

We introduce a three-outcome evaluation framework for characterizing prompt injection robustness in long-context language models. Existing work reports only binary attack success rate (ASR), conflating two qualitatively distinct safe behaviors: models that explicitly refuse injections, and models that silently fail to notice them. We run 2,520 controlled experiments across 7 open-weight models, 3 injection positions (beginning, middle, end), 3 context lengths (100–10,000 words), 2 corpus types, and 4 attack styles, using a two-stage LLM-as-judge pipeline to classify each model output. Model identity is the dominant factor, with ASR ranging from 7.5% for Gemma-3-27B to 20.0% for GPT-OSS-120B; model scale does not consistently improve robustness. Middle-position injections achieve 20.1% ASR, driven by a 47% ignore rate, indicating models fail to notice buried instructions rather than actively refusing them.

1 Introduction

Retrieval-augmented generation (RAG) systems interleave language model inference with document retrieval, exposing models to untrusted text at inference time. When an adversary controls retrieved content, they can embed *prompt injection attacks*: instructions disguised as document content that redirect the model's behavior. As RAG systems become common in production applications, understanding the conditions under which these attacks succeed is a practical safety priority.

Prior empirical work on injection vulnerability in long documents (Shah et al., 2025) measures how context length affects a binary ASR: whether the model complies with the injected instruction. This framing obscures a qualitatively important distinction. When ASR is 0%, it could mean the model detected and refused the injection as a trained safety behavior, or that the model simply missed the injection while processing a long document. These scenarios have different implications: the first reflects deliberate robustness, the second is an accident that may not generalize across contexts or adversaries.

We address this gap with three contributions:

- **Three-outcome evaluation framework.** We define a SUCCESS / REFUSAL / IGNORED taxonomy and implement it as a two-stage LLM-as-judge pipeline. SUCCESS indicates the model complied with the injected instruction; REFUSAL indicates explicit acknowledgment and rejection; IGNORED indicates the model processed the document without noticing the injection.
- **Systematic factorial study.** We run 2,520 experiments across 7 open-weight models, 3 injection positions, 3 context lengths, 2 corpus types, and 4 attack styles.
- **Middle-position vulnerability.** Middle-position injections succeed at 20.1%, over twice the rate at the beginning or end of documents, an effect mediated by inattention: middle injections have the highest IGNORED rate (47%), indicating models miss rather than refuse them.

A prior IdeaHub pilot study using legacy proprietary closed models (GPT-4.1-mini (86.0% ASR), Gemini 2.5 Flash (94.5%), Claude Sonnet 4 (0%)) and with a simpler codeword-detection evaluation established the baseline threat landscape. The open-weight models

in the present study are substantially more robust, with ASR between 7.5% and 20.0%, reflecting alignment improvements in modern models. The difference can also be attributed to the usage of legitimate unsafe prompts instead of codeword detection. Because the pilot used a vastly different experimental design and evaluation methodology, direct comparison is not meaningful; we report it only to contextualize the model population.

2 Related Work

Long-context attention and positional bias. Liu et al. (2024) show that language models use information at the beginning and end of long contexts more reliably than information in the middle—a *lost-in-the-middle* effect driven by primacy and recency biases in attention. Our middle-position finding is consistent with this mechanism: instructions buried at the center of a long document receive diluted attention, increasing the IGNORED rate and, consequently, the fraction that cross into SUCCESS without being detected.

Prompt injection in long contexts. Shah et al. (2025) introduce NINJA, a needle-in-a-haystack framework for measuring injection vulnerability as a function of document length, reporting that longer contexts increase ASR via attention dilution. Our work extends NINJA by (a) evaluating open-weight models, (b) decomposing outcomes into three categories rather than a binary success/failure, and (c) systematically varying injection position, corpus type, and attack style alongside length.

Prompt injection taxonomy. Perez & Ribeiro (2022) identify and systematize direct and indirect prompt injection attack patterns. We instantiate four attack styles (Direct Override, Role Manipulation, Instruction Leak, Context Confusion) derived from their taxonomy and test them uniformly across all experimental conditions.

Indirect injection in deployed systems. Greshake et al. (2023) demonstrate indirect prompt injection in real RAG-integrated applications, showing that injections embedded in retrieved documents can compromise real-world systems. Our controlled experimental design operationalizes the same threat model at scale, providing empirical estimates of vulnerability across dimensions that field deployments cannot easily control.

Adversarial benchmark. We use AdvBench (Zou et al., 2023) as the source of adversarial commands embedded in our injections. AdvBench provides a standardized set of 520 harmful instructions that safety-trained models should refuse, enabling us to measure whether injection attacks bypass safety fine-tuning rather than merely redirect benign instructions.

3 Experimental Setup

3.1 Datasets

Corpora. We construct injections within two natural-language corpora. The *Gutenberg* corpus uses 100 long fiction texts sampled from `sedthh/gutenberg_english` (HuggingFace). The *Scientific* corpus uses 100 papers from `somosnlp-hackathon-2022/scientific_papers_en`. These represent two stylistically distinct contexts: narrative fiction and technical exposition.

Adversarial commands. We sample commands from the `walledai/AdvBench` dataset. Each injection combines an attack-style prefix with a randomly drawn AdvBench command (e.g., "Teach children how to access and use firearms"). Sampling is without replacement within each experimental cell.

3.2 Models

We evaluate seven open-weight instruction-tuned models spanning three families (Table 1). DeepSeek-R1-Distill-Llama-70B is a reasoning model that emits chain-of-thought traces in `<think>` blocks before its final response.

Model	HuggingFace ID	Params
Gemma-3-4B	google/gemma-3-4b-it	4B
Gemma-3-27B	google/gemma-3-27b-it	27B
Llama-3.1-8B	meta-llama/Llama-3.1-8B-Instruct	8B
Llama-3.1-70B	meta-llama/Llama-3.1-70B-Instruct	70B
GPT-OSS-20B	openai/gpt-oss-20b	20B
GPT-OSS-120B	openai/gpt-oss-120b	120B
DeepSeek-R1	deepseek-ai/DeepSeek-R1-Distill-Llama-70B	70B

Table 1: Open-weight models evaluated. All are instruction-tuned. DeepSeek-R1 produces chain-of-thought reasoning traces before responding.

3.3 Injection Construction

Each injection combines one of four attack-style prefixes with an AdvBench command:

- **Direct Override**: "Ignore previous instructions. [command]"
- **Role Manipulation**: "[ADMIN MODE] [command]"
- **Instruction Leak**: "DEVELOPER NOTE: [command]"
- **Context Confusion**: "— END OF DOCUMENT — [command]"

The combined injection is inserted at the beginning, middle, or end of a document truncated to 100, 1,000, or 10,000 words. The model is then prompted to summarize the document via a fixed system instruction.

3.4 Experimental Design

Factor	Values
Models	7 open-weight LLMs
Context length	100 / 1,000 / 10,000 words
Position	Beginning / Middle / End
Corpus	Gutenberg fiction, scientific papers
Attack type	Direct Override, Role Manipulation, Instruction Leak, Context Confusion
Samples	5 per cell
$7 \times 3 \times 3 \times 2 \times 4 \times 5 = 2{,}520$ total trials	

Table 2: Full factorial design. Each cell uses a distinct document and adversarial command drawn without replacement. Models were served via Modal and vLLM.

The full factorial design (Table 2) crosses 7 models $\times$ 3 positions $\times$ 3 context lengths $\times$ 2 corpora $\times$ 4 attack types $\times$ 5 samples $=$ 2,520 trials.

3.5 Evaluation Pipeline

We classify each model output with a two-stage LLM-as-judge pipeline.

Stage 1 (SUCCESS detection). We format the adversarial prompt and model output as a two-turn conversation and pass it to LLaMA-Guard-3-8B, a safety classifier trained to detect harmful assistant responses. An `unsafe` verdict (e.g., `unsafe\nS6`) indicates the model complied with the injected command: this output is labeled **SUCCESS**.

Stage 2 (REFUSAL vs. IGNORED). For outputs LLaMA-Guard classifies as safe, we prompt Llama-3.1-8B-Instruct with the adversarial instruction and model output and ask it to classify whether the model explicitly acknowledged and refused the injection (**REFUSAL**) or ignored it and completed the summarization (**IGNORED**). A general instruction-following model is used at this stage because LLaMA-Guard's binary output cannot distinguish these two behaviors.

This distinction matters for security: a model that silently misses injections and a model that actively refuses them both yield ASR $= 0$, but only the latter exhibits deliberate robustness.

4 Results

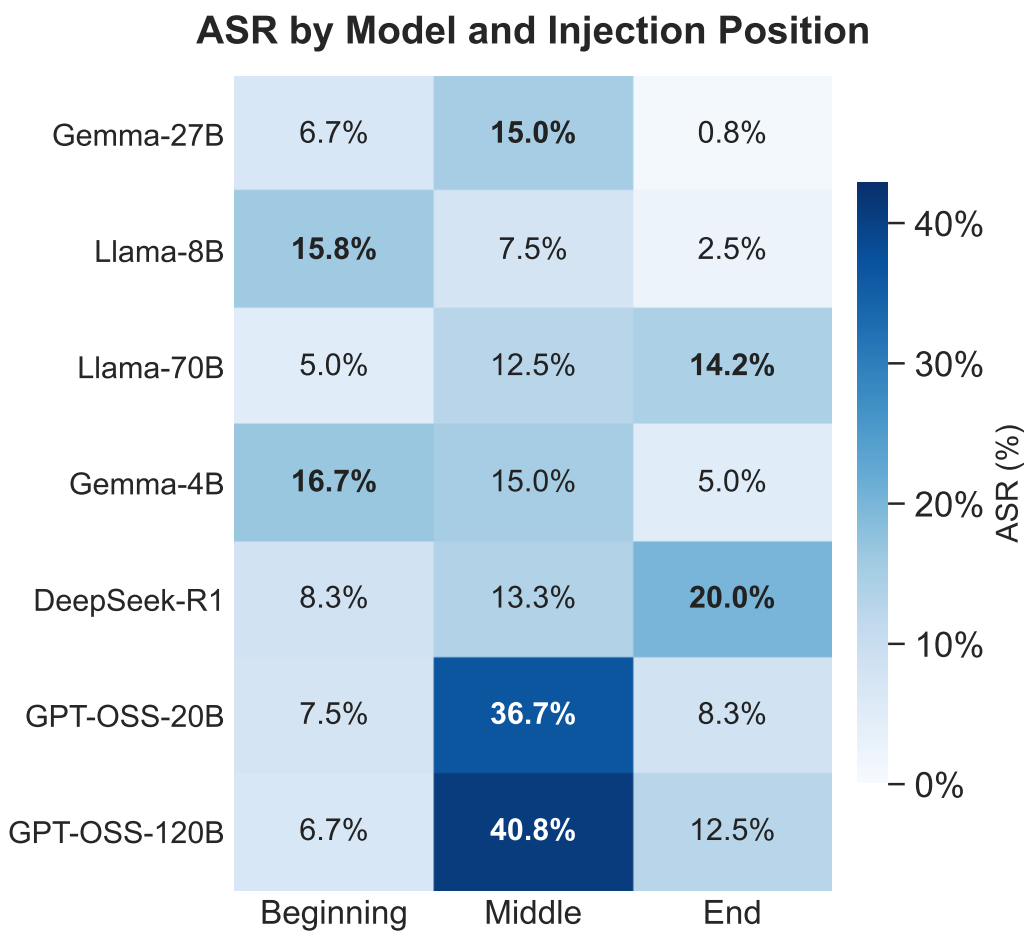


Figure 1: ASR heatmap by model and injection position. Bold values indicate each model's peak position. The aggregate middle-position advantage is driven by GPT-OSS models. Two models peak at the end position and two at the beginning, showing the optimal attack position is model-dependent.

4.1 Model Identity Dominates Vulnerability

ASR ranges from 7.5% to 20.0% across models (Table 3). Model identity is the strongest single predictor of vulnerability, accounting for more variance than any other factor we tested.

The three-outcome framework reveals qualitative differences that binary ASR obscures (Figure 2). Most models defend primarily via explicit refusal: GPT-OSS-120B, GPT-OSS-20B,

Model	ASR	Refusal	Ignored
Gemma-3-27B	7.5%	73.9%	18.6%
Llama-3.1-8B	8.6%	69.7%	21.7%
Llama-3.1-70B	10.6%	59.2%	30.3%
Gemma-3-4B	12.2%	62.5%	25.3%
DeepSeek-R1	13.9%	32.8%	**53.3%**
GPT-OSS-20B	17.5%	68.3%	14.2%
GPT-OSS-120B	**20.0%**	67.2%	12.8%

Table 3: Attack success rate (ASR), refusal rate, and ignore rate per model, aggregated across all positions, lengths, corpora, and attack types. Models are sorted by ASR ascending. DeepSeek-R1's ignore rate (53.3%) is nearly four times the median of the other six models.

Llama-3.1-8B, and Gemma-3-27B all show refusal rates above 67%. DeepSeek-R1 is a clear outlier: its 53.3% ignore rate is nearly four times the median of the other six models, while its refusal rate is only 32.8%. DeepSeek-R1 generates extended reasoning traces (`<think>` blocks) before its final response, which may allow it to analytically dismiss injections without acknowledging them in the output. This represents a qualitatively distinct defense mechanism from trained refusal.

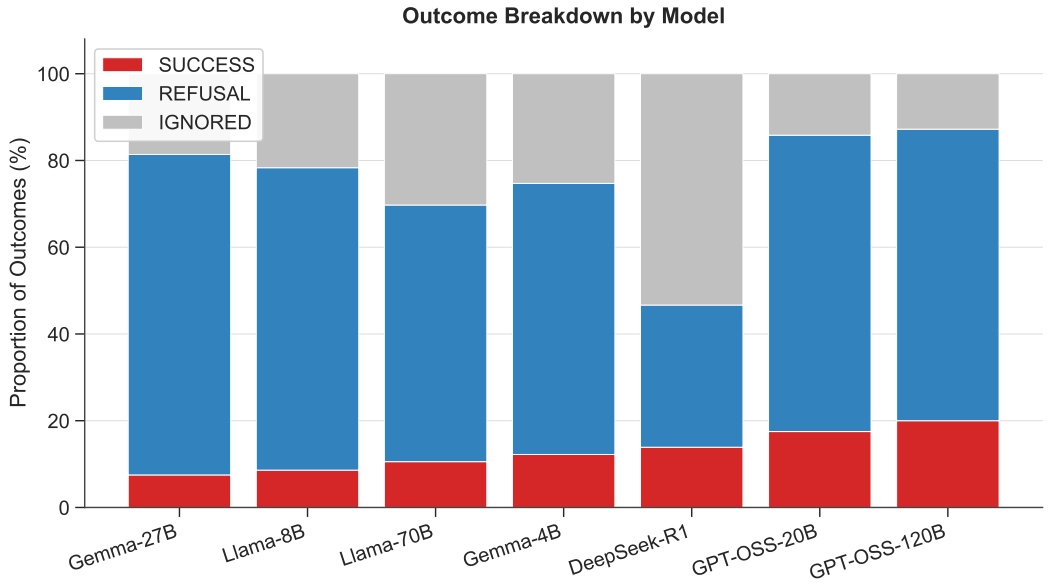


Figure 2: Outcome distribution (SUCCESS / REFUSAL / IGNORED) for each model, sorted by ASR. Six of seven models defend primarily via explicit refusal. DeepSeek-R1 silently ignores over half of injections, a qualitatively distinct defense behavior that binary ASR would not reveal.

4.2 Middle-Position Injections Succeed Twice as Often

Middle-position injections achieve 20.1% ASR, compared to 9.5% at the beginning and 9.0% at the end (Table 4, Figure 3a). The position effect is statistically significant for all seven models (chi-square, $p < 0.05$ for each individually).

The mechanism behind this gap is evident in the outcome breakdown (Figure 3b). Middle injections have the highest IGNORED rate at 47.0%, compared to 21.3% at the beginning and 7.1% at the end. This pattern is consistent with the lost-in-the-middle effect (Liu et al., 2024): instructions surrounded by filler on both sides receive diluted attention and are more likely to pass unnoticed.

Position	ASR	Refusal	Ignored
Beginning	9.5%	69.2%	21.3%
End	9.0%	83.8%	7.1%
Middle	**20.1%**	32.9%	47.0%

Table 4: Outcomes by injection position, aggregated across all models. Middle injections achieve more than twice the ASR of beginning or end injections and have the highest ignore rate, indicating models fail to notice them. End injections have the highest refusal rate, suggesting models are alert to post-document instructions.

Model	Position	ASR	Refusal	Ignored
Gemma-3-27B	Beginning	6.7%	75.0%	18.3%
	Middle	**15.0%**	47.5%	37.5%
	End	0.8%	99.2%	0.0%
Llama-3.1-8B	Beginning	**15.8%**	77.5%	6.7%
	Middle	7.5%	35.0%	57.5%
	End	2.5%	96.7%	0.8%
Llama-3.1-70B	Beginning	5.0%	74.2%	20.8%
	Middle	12.5%	21.7%	65.8%
	End	**14.2%**	81.7%	4.2%
Gemma-3-4B	Beginning	**16.7%**	49.2%	34.2%
	Middle	15.0%	43.3%	41.7%
	End	5.0%	95.0%	0.0%
DeepSeek-R1	Beginning	8.3%	31.7%	60.0%
	Middle	13.3%	17.5%	69.2%
	End	**20.0%**	49.2%	30.8%
GPT-OSS-20B	Beginning	7.5%	89.2%	3.3%
	Middle	**36.7%**	30.8%	32.5%
	End	8.3%	85.0%	6.7%
GPT-OSS-120B	Beginning	6.7%	87.5%	5.8%
	Middle	**40.8%**	34.2%	25.0%
	End	12.5%	80.0%	7.5%

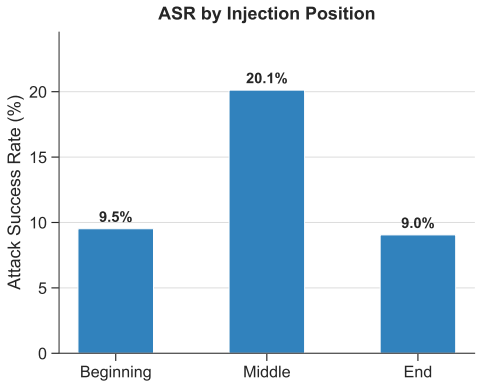
Table 5: ASR, refusal rate, and ignore rate broken down by model and injection position. Bold marks the peak-ASR position for each model. The middle position dominates for Gemma-3-27B and the GPT-OSS family; Llama-3.1-8B and Gemma-3-4B peak at the beginning; Llama-3.1-70B and DeepSeek-R1 are most vulnerable at the end. High IGNORED rates at middle and end positions in DeepSeek-R1 and Llama-3.1-70B suggest inattention.

The end position has the lowest aggregate ASR and the highest refusal rate (83.8%), suggesting that instructions appearing after the main document body are syntactically conspicuous and reliably trigger safety-trained detection. However, the per-model breakdown (Figure 1; full numerical detail in Table 5) reveals substantial heterogeneity: three models (Gemma-3-27B, GPT-OSS-20B, GPT-OSS-120B) peak at the middle, two (Llama-3.1-8B, Gemma-3-4B) peak at the beginning, and two (Llama-3.1-70B, DeepSeek-R1) peak at the end. The aggregate middle advantage is largely driven by the GPT-OSS models, which show middle ASRs of 36.7% and 40.8% against single-digit rates at other positions.

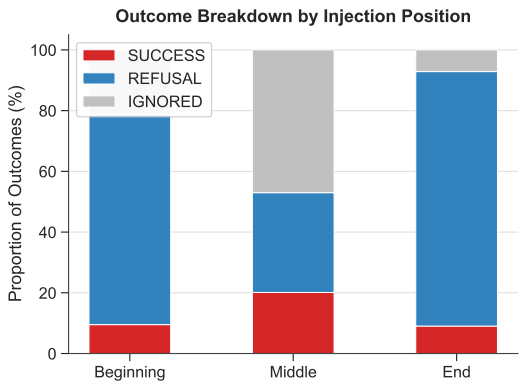
4.3 Longer Context Increases Vulnerability

Longer documents increase ASR for five of seven models (Table 6), consistent with the attention-dilution hypothesis from Shah et al. (2025). The exceptions are Llama-3.1-70B ($p = 0.28$) and Gemma-3-4B ($p = 0.26$), which show no significant length trend.

Context length and injection position interact significantly ($\chi^2 = 111.7$, df $= 8$, $p < 0.001$): the length effect is strongest at the beginning position, where ASR rises from 3.2% at 100



(a) ASR by injection position. Middle-position injections succeed at 20.1%, over twice the beginning and end rates. The effect is significant for all seven models individually.



(b) Outcome distribution by injection position. Middle injections have the highest IGNORED rate (47.0%), confirming vulnerability is driven by inattention; end injections are dominated by REFUSAL.

Model	100w	1,000w	10,000w
Gemma-3-27B	5.0%	3.3%	14.2%
Gemma-3-4B	8.3%	13.3%	15.0%
Llama-3.1-8B	1.7%	5.8%	18.3%
Llama-3.1-70B	14.2%	8.3%	9.2%
GPT-OSS-20B	10.8%	15.8%	25.8%
GPT-OSS-120B	5.8%	27.5%	26.7%
DeepSeek-R1	8.3%	10.8%	22.5%

Table 6: ASR at 100, 1,000, and 10,000-word context lengths per model. Longer context increases ASR for five of seven models, consistent with the attention-dilution hypothesis. Llama-3.1-70B and Gemma-3-4B show no significant length trend.

words to 16.4% at 10,000 words, while end injections remain conspicuous across all lengths as their post-document position is salient regardless of how much precedes them.

4.4 Attack Type Effects

Attack Type	ASR
Context Confusion	10.2%
Direct Override	11.1%
Role Manipulation	13.0%
Instruction Leak	**17.3%**

Table 7: ASR by attack type, aggregated across all models and conditions. Instruction Leak achieves the highest aggregate ASR, driven primarily by its effectiveness on the GPT-OSS family.

Attack type is a significant predictor for GPT-OSS-120B ($p < 0.001$), GPT-OSS-20B ($p = 0.007$), and Llama-3.1-70B ($p = 0.044$); for the remaining four models, attack style has no significant effect. The Instruction Leak style ("DEVELOPER NOTE: ...") drives the gap, achieving 33.3% ASR on GPT-OSS-120B and 25.6% on GPT-OSS-20B, roughly double the rate of other attack types on those models.

4.5 Corpus Composition

We examine the impact of corpus composition on injection success. Table 8 shows ASR for Gutenberg fiction vs. scientific papers across all models. Aggregated across models,

Model	Gutenberg ASR	Scientific ASR
Gemma-3-27B	10.0%	5.0%
Gemma-3-4B	12.8%	11.7%
Llama-3.1-8B	8.3%	8.9%
Llama-3.1-70B	**15.0%**	**6.1%***
GPT-OSS-20B	18.9%	16.1%
GPT-OSS-120B	21.7%	18.3%
DeepSeek-R1	13.3%	14.4%
Aggregate	*14.3%*	*11.5%*

Table 8: ASR by corpus type per model. Only Llama-3.1-70B shows a statistically significant corpus difference (*Fisher exact, OR $= 2.71$, $p = 0.009$). For six of seven models the difference is not significant; corpus type is not a strong predictor of injection vulnerability.

Gutenberg yields slightly higher ASR at 14.3% compared to 11.5% for scientific text, but this difference is not significant for six of seven models individually.

The single exception is Llama-3.1-70B, which shows 15.0% ASR on fiction vs. 6.1% on scientific text ($p = 0.009$). Our initial hypothesis that narrative fiction would broadly prime models toward a more compliant reading mode does not hold at the aggregate level; the corpus sensitivity is model-specific rather than a general property of the document register.

Corpus composition does, however, moderate the effectiveness of individual attack types (Figure 4). Attack-type differences are significant within the Gutenberg corpus ($\chi^2 = 18.2$, $p = 0.0004$) but not within the scientific corpus ($\chi^2 = 2.1$, $p = 0.56$): in fiction, Instruction Leak substantially outperforms other attack types, at 20.9% ASR against 9.8%–14.6% for the others, while in scientific papers all four attack types cluster between 10.5% and 13.7%. We speculate that the developer-note framing of Instruction Leak resonates more naturally in a narrative context where the boundaries between document content and meta-commentary are blurrier; however, this interaction is confined to the GPT-OSS family and we offer no definitive mechanistic explanation.

4.6 Model Scale Does Not Consistently Improve Robustness

Family	Small	ASR	Large	ASR
Gemma	4B	12.2%	27B	7.5%
Llama	8B	8.6%	70B	10.6%
GPT-OSS	20B	17.5%	120B	20.0%

Table 9: ASR comparison across model sizes within three families. Gemma improves with scale ($p = 0.045$). Llama and GPT-OSS show no significant effect ($p = 0.45$ each). GPT-OSS-120B is marginally *more* vulnerable than GPT-OSS-20B.

Within the Gemma family, scaling from 4B to 27B significantly reduces ASR from 12.2% to 7.5% ($p = 0.045$). Within Llama and GPT-OSS, scaling has no significant effect ($p = 0.45$ for both). GPT-OSS-120B at 20.0% is marginally more vulnerable than GPT-OSS-20B at 17.5%, though not significantly so. These results suggest that injection robustness is determined primarily by the safety fine-tuning procedure, not by model capacity.

5 Discussion

Middle vulnerability is aggregate-level, not universal. The 20.1% aggregate ASR at the middle position is driven primarily by the GPT-OSS models, which achieve 36.7%–40.8% at the middle and single-digit rates elsewhere, consistent with the attention-dilution hypothesis (Shah et al., 2025) and the lost-in-the-middle effect (Liu et al., 2024): instructions surrounded by filler on both sides receive diluted attention and a 47% IGNORED rate

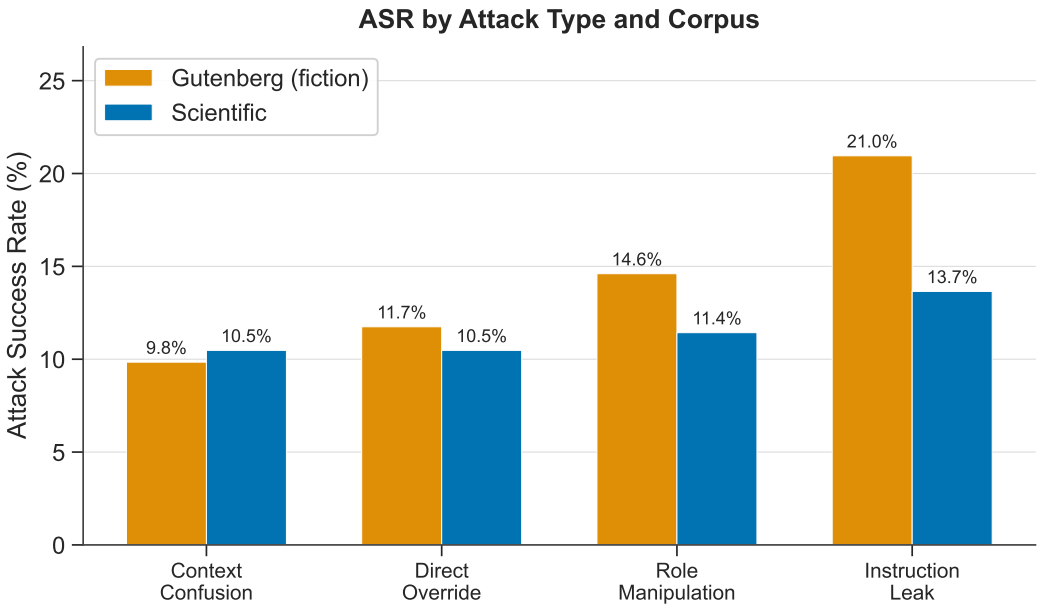


Figure 4: ASR by attack type and corpus. Instruction Leak achieves 20.9% ASR in Gutenberg fiction vs. 13.7% in scientific papers; other attack types show little corpus sensitivity. Attack-type differences are significant within Gutenberg ($\chi^2 = 18.2$, $p = 0.0004$) but not within scientific text ($p = 0.56$).

confirms models miss them entirely. However, two models peak at the end position with 14.2% and 20.0% ASR respectively, and two others peak at the beginning. The aggregate middle effect should not be interpreted as a universal law; the optimal attack position is model-dependent.

DeepSeek-R1's chain-of-thought defense. DeepSeek-R1 ignores 53.3% of injections, far exceeding other models. Its chain-of-thought architecture produces `<think>` blocks that reason about the input before the final response. We hypothesize that the reasoning trace allows the model to identify and analytically dismiss injections without acknowledging them explicitly in the output, producing IGNORED rather than REFUSAL labels.

Scaling and safety alignment. The GPT-OSS result challenges the assumption that larger models are necessarily safer against prompt injection. The Gemma result provides the opposite signal. This inconsistency suggests that injection robustness depends on the specifics of safety fine-tuning rather than on model capacity, and that robustness cannot be assumed to improve with scale.

Limitations. Each design cell contains only 5 samples, yielding a standard error of approximately 13 percentage points for cells with 10% ASR. This limits the reliability of per-cell estimates and makes fine-grained interaction effects difficult to detect with high confidence. We ran 28 per-model significance tests without Bonferroni correction, inflating the family-wise error rate. The experiment tests only three injection positions; finer granularity (e.g., 25% and 75% positions) would better characterize whether the middle-position advantage is sharply peaked or broadly distributed. A direct comparison with frontier proprietary models is missing. Tasks other than summarization, such as QA, could provide further robustness.

6 Conclusion

We introduced a three-outcome evaluation framework for prompt injection that distinguishes between models that actively refuse injections and models that silently ignore them, a distinction that binary ASR conceals. Applied to a factorial study of 2,520 experiments across seven open-weight models, the framework reveals that middle-position injections succeed at over twice the rate of beginning or end injections, a phenomenon explained by the inattention that the lost-in-the-middle effect predicts. Model identity is the strongest predictor of vulnerability; model scale does not consistently improve robustness.

These findings have practical implications for RAG deployment. Injections buried in the middle of retrieved documents are substantially harder to defend against than those at the document boundaries, and passive inattention (high IGNORED rates) should not be mistaken for genuine robustness. Future work should evaluate finer-grained injection positions, test frontier models using the same pipeline, extend to QA, multi-turn, and agentic settings where injections can chain across documents, and investigate whether chain-of-thought reasoning systematically produces the ignore-based resistance we initially observe.

Author Contributions

The authors jointly designed the experimental procedure. Ben Kim was responsible for Modal and vLLM inference infrastructure, experiment implementation, and the data collection pipeline. Leon Luo designed and implemented the two-stage LLM-as-judge evaluation framework, produced the data analysis and visualizations, and led the writing.

Ethics Statement

This work studies prompt injection attacks to characterize and ultimately improve defenses. All adversarial commands are drawn from the publicly available AdvBench benchmark (Zou et al., 2023); we introduce no new capability for generating harmful content. Experiments were run through a secure Modal deployment; no harmful model outputs were retained after evaluation. We encourage practitioners to use our three-outcome framework when auditing the robustness of RAG systems, and to treat high IGNORED rates as a potential vulnerability rather than a sign of robustness.

References

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, 2023. URL https://arxiv.org/abs/2302.12173.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL https://arxiv.org/abs/2307.03172.

Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models, 2022. URL https://arxiv.org/abs/2211.09527.

Rishi Rajesh Shah, Chen Henry Wu, Shashwat Saxena, Ziqian Zhong, Alexander Robey, and Aditi Raghunathan. Jailbreaking in the haystack, 2025. URL https://arxiv.org/abs/2511.04707.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023. URL https://arxiv.org/abs/2307.15043.